SYNAPTYC

by nano-SYNAPSYS

Architecture Design Document

End-to-End Encrypted Messaging Platform

Version 1.3.1 | March 2026 | Confidential
Platform: React Native (Expo) | Backend: Elixir/Phoenix | Storage: Cloudflare R2

Table of Contents

1. Executive Summary
2. System Architecture Diagram
3. Component Hierarchy
4. Authentication & Session Management
5. End-to-End Encryption (E2EE)
6. Media Encryption & Storage
7. Real-Time Messaging (WebSocket)
8. Local Database & Sync
9. Banner AI Assistant
10. Backend API Reference
11. Security Model
12. Third-Party Integrations
13. Data Flow Diagrams

1 Executive Summary

SYNAPTYC is a privacy-first, end-to-end encrypted messaging application built with React Native (Expo). It implements the **Signal Protocol** (X3DH + Double Ratchet) for 1-to-1 messaging and **Sender Keys** for group encryption, ensuring forward secrecy and post-compromise security.

Key Capabilities

- **E2EE Messaging** — Signal Protocol (X3DH + Double Ratchet)
- **Group Encryption** — Sender Key distribution
- **Encrypted Media** — AES-256-GCM + Cloudflare R2
- **Offline-First** — SQLCipher local DB with delta sync
- **Real-Time** — Phoenix WebSocket channels
- **Biometric Auth** — Face ID + PIN + device binding
- **AI Assistant** — Banner with permission-gated tools
- **Disappearing Messages** — 1 min to 30 days
- **Push Notifications** — Expo Push + local alerts
- **Contact Integration** — Phone contacts + SMS invite
- **Themes** — Green (hacker) & Tactical (military)
- **Deep Linking** — Invite tokens via custom scheme

Technology Stack

Layer	Technology	Purpose
Client	React Native 0.81 + Expo 54	Cross-platform mobile app
Backend	Elixir / Phoenix	REST API + WebSocket channels
Database (server)	PostgreSQL	User data, messages, keys
Database (client)	SQLCipher (op-sqlite)	Encrypted local message cache
Object Storage	Cloudflare R2	Encrypted media blobs
Crypto (DH)	tweetnacl (NaCl)	X25519 key exchange
Crypto (KDF)	@noble/hashes	HKDF, HMAC-SHA256
Crypto (media)	WebCrypto (AES-256-GCM)	Media file encryption
Key Storage	expo-secure-store	Encrypted device keychain
Push	Expo Notifications	Push token management

2 System Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                          SYNAPTYC CLIENT (Expo / React Native)               │
│                                                                              │
│   ┌─────────┐  ┌─────────┐  ┌─────────┐  ┌─────────┐  ┌─────────┐           │
│   │  CHATS  │  │ GROUPS  │  │ BANNER  │  │ PROFILE │  │ SETTINGS │          │
│   │   Tab   │  │   Tab   │  │ AI Tab  │  │   Tab   │  │   Tab   │           │
│   └─────────┘  └─────────┘  └─────────┘  └─────────┘  └─────────┘           │
│        │            │            │            │            │                 │
│   ┌────▼────┐  ┌────▼────┐       │       ┌────▼────┐                        │
│   │ DM Chat │  │ Group   │       │       │ Image   │                        │
│   │ Screen  │  │ Chat Scr │      │       │ Picker  │                        │
│   └─────────┘  └─────────┘       │       └─────────┘                        │
│        │            │            │            │                             │
│   ┌────▼────────────▼────────────▼────────────▼──────────────────────────┐   │
│   │                ENCRYPTION LAYER (signal_protocol.js)                 │   │
│   │  ┌──────────┐  ┌──────────────┐  ┌──────────────┐                  │   │
│   │  │  X3DH    │  │ Double       │  │ Sender Keys  │                  │   │
│   │  │ Handshake│  │ Ratchet (DM) │  │ (Groups)     │                  │   │
│   │  └──────────┘  └──────────────┘  └──────────────┘                  │   │
│   │  tweetnacl (X25519)  |  @noble/hashes (HKDF, HMAC-SHA256)          │   │
│   └──────────────────────────────────────────────────────────────────────┘   │
│                                  │                                           │
│   ┌──────────────────────────────▼──────────────────────────────────────┐   │
│   │                        TRANSPORT LAYER                               │   │
│   │  ┌──────────────────┐    ┌──────────────────┐                      │   │
│   │  │ Phoenix WebSocket │    │  REST API (fetch) │                     │   │
│   │  │ (real-time msgs)  │    │  (fallback/media) │                     │   │
│   │  └──────────────────┘    └──────────────────┘                      │   │
│   └──────────────────────────────────────────────────────────────────────┘   │
│                                  │                                           │
│   ┌──────────────────────────────▼──────────────────────────────────────┐   │
│   │                        LOCAL STORAGE                                 │   │
│   │  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐              │   │
│   │  | SQLCipher DB |  | SecureStore   |  | FileSystem   |              │   │
│   │  | (messages,   |  | (keys, tokens |  | (profile images |           │   │
│   │  | sync cursors)|  | sessions)    |  | media cache) |              │   │
│   │  └──────────────┘  └──────────────┘  └──────────────┘              │   │
│   └──────────────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────────┘
                  |  WSS (encrypted)        |  HTTPS (encrypted)
                  ▼                         ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                    BACKEND SERVER (Elixir / Phoenix)                         │
│                    nano-synapsys-server.fly.dev                              │
│                                                                              │
│   ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────┐        │
│   | Auth Module  |  | Messaging    |  | Signal Key   |  | Banner AI |       │
│   | (PIN, JWT,   |  | (DMs, Groups |  | Store        |  | (LLM      |       │
│   | Bio-refresh) |  | Delivery)    |  | (PreKeys,    |  | Endpoint)|        │
│   └──────────────┘  └──────────────┘  | SenderKeys)|  └──────────┘        │
│          |                |           └──────────┘          |               │
│   ┌──────▼────────────────▼──────────────────────────────────▼──────────┐   │
│   |                        PostgreSQL Database                           |   │
│   | users | messages | groups | signal_prekeys | sender_keys | devices |   │
│   └──────────────────────────────────────────────────────────────────────┘   │
│                                                                              │
│   ┌──────────────────────────┐   ┌──────────────────────────┐               │
│   | Phoenix Channels (WS)    |   | Presigned URL Generator  |               │
│   | room:{peerId}            |   | → Cloudflare R2          |               │
│   | group:{groupId}          |   └──────────────────────────┘               │
│   └──────────────────────────┘                  |                           │
└──────────────────────────────────────────────────|──────────────────────────┘
                                                   |
                                                   ▼
                                   ┌──────────────────────────┐
                                   |    Cloudflare R2         |
                                   |    (Encrypted Media Blobs) |
```

```
                                        │    AES-256-GCM ciphertext   │
                                        └─────────────────────────────┘
```

3 Component Hierarchy

```
App()
└─ SkinProvider                        ← Theme context (green | tactical)
   └─ AppInner                         ← Auth gate + session management
      ├─ [loading]   → Splash screen
      ├─ [auth]      → AuthScreen      ← PIN login / register / reset
      ├─ [biometric] → BiometricUnlock ← Face ID prompt
      └─ [home]      → HomeScreen      ← Main app (tabs + WS)
         │
         ├─ TabBar  [CHATS | GROUPS | BOT | PROFILE | SETTINGS]
         │
         ├─ ChatsTab                   ← DM contact grid
         │  ├─ PhoneContactsSheet      ← Invite contacts
         │  └─ → DMChatScreen          ← 1:1 encrypted conversation
         │
         ├─ GroupsTab                  ← Group grid + create form
         │  └─ → GroupChatScreen       ← Group encrypted conversation
         │     └─ GroupMembersModal    ← Member list
         │
         ├─ BotTab                     ← Banner AI chat interface
         │
         ├─ ProfileTab                 ← Avatar, bio, Face ID setup
         │
         └─ SettingsTab               ← Notifications, disappear, theme
```

4 Authentication & Session Management

4.1 Authentication Flow

```
┌──────────────┐      ┌──────────────┐      ┌──────────────────┐
│  App Launch  │──────►│ Check Token  │──────►│ Token exists?    │
└──────────────┘      │ (SecureStore)│      └──────────────────┘
                      └──────────────┘                 │
                                                       │
                                  ┌────────────────────┴────────┐
                                  │ NO                          │ YES
                                  ▼                             ▼
                        ┌──────────────────┐      ┌──────────────────┐
                        │   Auth Screen    │      │ Check skip-auth  │
                        │   (PIN Entry)    │      │ (24h window?)    │
                        └──────────────────┘      └──────────────────┘
                                 │                          │
                                 │                          │
                     ┌───────────┴──────────┐   ┌──────────┴──────────┐
                     │ LOGIN        │ REGISTER  │ YES      │ NO
                     ▼              ▼           ▼          ▼
┌──────────────────┐ ┌──────────┐ ┌──────────┐ ┌──────────────┐
│POST /auth/       │ │POST /auth│ │bio-refresh│ │Bio enabled?  │
│  pin-login       │ │ /register│ │(silent)  │ └──────────────┘
│{device_id, pin}  │ │{username,│ └──────────┘   YES  │  NO
└──────────────────┘ │ device_id│        │          ▼    ▼
        │            │ pin}     │        │
        │            └──────────┘        │   ┌──────────┐ ┌──────┐
        │                │               │   │ Face ID  │ │ Home │
        │                │               │   │ Prompt   │ │Screen│
        ▼                ▼               ▼   └──────────┘ └──────┘
                                                  │
┌─────────────────────────────────────────────────┴──────────────┐
│                    { token, user, refresh }                    │
│  → SecureStore: JWT_KEY, USER_KEY, BIO_REFRESH_KEY             │
│  → _setSession(token, refresh, updater)                        │
│  → Navigate to HomeScreen                                      │
└────────────────────────────────────────────────────────────────┘
```

4.2 JWT Auto-Refresh

Every API call checks for `401 Unauthorized`. On expiry, the client automatically refreshes:

1. Send stored refresh token to `POST /auth/bio-refresh`
2. Receive new access token + refresh token
3. Persist both to SecureStore
4. Retry the original request (one retry only)

4.3 Device Binding

Each device generates a **UUID v4** on first launch, stored permanently in SecureStore. All authentication is tied to this device ID — a PIN only works on the device where it was registered.

4.4 Secure Storage Keys

Key	Value	Purpose
`nano_jwt`	JWT access token	API authentication
`nano_user`	User JSON object	Current user data
`nano_bio_refresh`	Refresh token	Bio/silent token refresh
`nano_bio_enabled`	'1' or '0'	Face ID toggle
`nano_device_id`	UUID v4	Permanent device identity
`nano_skip_auth`	Expiry timestamp	24h skip auth window
`nano_profile_image`	Local file URI	Profile avatar path
`nano_skin`	'green' or 'tactical'	UI theme preference
`nano_disappear`	Seconds (null/60/...)	Disappearing message timer

5 End-to-End Encryption (E2EE)

5.1 Cryptographic Primitives

Primitive	Algorithm	Library	Usage
Key Exchange	X25519 (Curve25519)	tweetnacl	DH in X3DH & Double Ratchet
Symmetric Encryption	XSalsa20-Poly1305	tweetnacl	Message ciphertext
KDF	HKDF-SHA256	@noble/hashes	Root key & chain key derivation
MAC	HMAC-SHA256	@noble/hashes	Chain key advance, pre-key signing
Media Encryption	AES-256-GCM	WebCrypto	Image/file encryption
RNG	CSPRNG	react-native-get-random-values	Key & nonce generation

5.2 Key Hierarchy

```
IDENTITY KEY (IK)                    ← Long-term, generated once per device
│                                       Stored: sig_identity_key_v1
│
├── SIGNED PRE-KEY (SPK)             ← Medium-term, rotatable
│   │  Signed by IK via HMAC-SHA256    Stored: sig_signed_prekey_v1
│   │  Uploaded to server
│   │
│   └── ONE-TIME PRE-KEYS (OPK)      ← Ephemeral, batch of 50
│       Uploaded to server              Stored: sig_one_time_prekeys_v1
│       Server maintains pool ≥ 20      Consumed on first contact
│
├── EPHEMERAL KEY (EK)                ← Generated per X3DH session
│   │  Never stored (used and discarded)
│   │
│   └── SESSION (Double Ratchet)      ← Derived from X3DH shared secret
│       ├── Root Key (RK)               Ratchets on each DH exchange
│       ├── Sending Chain Key (CKs)     Advances per message sent
│       ├── Receiving Chain Key (CKr)   Advances per message received
│       └── Message Key (MK)            One-time use, derived from CK
│
└── SENDER KEY (Group)                ← Per-group, per-member
    ├── Chain Key                       Advances per group message
    └── Distributed via DM sessions    Encrypted with Double Ratchet
```

5.3 X3DH Key Agreement (Session Init)

```
ALICE (initiator)                              BOB (responder)
────────────────────                           ────────────────────

Has: IK_A (identity)                           Has: IK_B (identity)
Generates: EK_A (ephemeral)                         SPK_B (signed pre-key)
                                                    OPK_B (one-time, optional)


1. Alice fetches Bob's pre-key bundle:
   GET /api/signal/prekeys/{bobId}
   → { identity_key, signed_prekey, one_time_prekey }

2. Alice computes 3-4 DH exchanges:
   DH1 = X25519(IK_A.secret,  SPK_B.public)   ← Identity × Signed PreKey
   DH2 = X25519(EK_A.secret,  IK_B.public)    ← Ephemeral × Identity
   DH3 = X25519(EK_A.secret,  SPK_B.public)   ← Ephemeral × Signed PreKey
   DH4 = X25519(EK_A.secret,  OPK_B.public)   ← Ephemeral × One-Time (opt.)

3. Derive shared secret:
   SK = HKDF(DH1 || DH2 || DH3 [|| DH4], salt=0, info='SYNAPTYC_X3DH', len=64)

4. Initialize Double Ratchet with SK as root key
   → Alice: sender ratchet (DHs = EK_A, DHr = SPK_B)

5. First message includes X3DH metadata:
   { ik: IK_A.pub, ek: EK_A.pub, spk_id, opk_id }

6. Bob receives → runs X3DH responder with same DH exchanges
   → Bob: receiver ratchet (DHs = SPK_B, DHr = EK_A)
```

5.4 Double Ratchet (DM Messages)

```
SENDING A MESSAGE                          RECEIVING A MESSAGE
────────────────────                       ────────────────────
1. Derive message key:                     1. Parse header: { spk, n, pn }
   MK = HMAC(CKs, 0x01)
   CKs_next = HMAC(CKs, 0x02)             2. If sender DH key changed:
                                               - Store skipped keys (up to 1000)
2. Encrypt:                                    - DH ratchet step:
   nonce = random(24 bytes)                      dh_out = X25519(DHs, new_DHr)
   ct = XSalsa20-Poly1305(MK, nonce, pt)        [RK, CKr] = HKDF(dh_out, RK)
                                                 Generate new DHs keypair
3. Build header:                                 [RK, CKs] = HKDF(X25519(new_DHs,
   { spk: DHs.pub, n: msgIdx, pn: prev }                          DHr), RK)

4. Wrap in Signal envelope:                3. Derive message key:
   { sig:true, v:1,                           MK = HMAC(CKr, 0x01)
     hdr: JSON(header),                       CKr_next = HMAC(CKr, 0x02)
     ct: base64(nonce + ct) }
                                           4. Decrypt:
5. Send via WS or REST                        pt = XSalsa20-Poly1305.open(MK,
                                                   nonce, ct)

ROOT KEY RATCHET:
────────────────────
KDF_RK(rk, dh_out) = HKDF(dh_out, rk, 'SYNAPTYC_DR_ROOT', 64)
                     → first 32 bytes = new Root Key
                     → last 32 bytes  = new Chain Key
```

5.5 Sender Keys (Group Messages)

```
GROUP KEY DISTRIBUTION
──────────────────────

1. Creator generates sender key:
   chainKey = random(32 bytes), iteration = 0

2. For each group member:
   encrypted_sk = DM_Encrypt(mySession[member], serialize(senderKey))

3. Upload to server:
   POST /api/signal/sender-keys { group_id, keys: [{recipient_id, encrypted_sk}] }

4. Members fetch & decrypt:
   GET /api/signal/sender-keys/{groupId}
   For each: senderKey = DM_Decrypt(session[sender], encrypted_sk)

GROUP MESSAGE ENCRYPTION
────────────────────────

1. Advance chain: MK = HMAC(chainKey, 0x01), chainKey = HMAC(chainKey, 0x02)
2. Encrypt: nonce = random(24), ct = XSalsa20-Poly1305(MK, nonce, plaintext)
3. Envelope: { gsig:true, v:1, sid:myId, hdr:iteration_b64, ct:nonce+ct_b64 }
4. All members decrypt with sender's cached chain key (advance to iteration)
```

6 Media Encryption & Storage

6.1 Encrypted Media Upload Flow

```
┌──────────┐    ┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│  Image   │    │  AES-256-GCM │    │  Presign URL │    │  Cloudflare  │
│  Picker  │──►│  Encrypt     │──►│  from Server │──►│  R2 Upload   │
│ (base64) │    │  (WebCrypto) │    │  POST /media │    │  PUT (presign│
└──────────┘    └──────────────┘    │  /presign    │    │  no auth)    │
                       │            └──────────────┘    └──────────────┘
                       │                                       │
                       ▼                                       ▼
                ┌──────────────┐                    ┌──────────────────┐
                │ key + iv     │                    │ public_url       │
                │ (both base64)│                    │ (R2 download URL)│
                └──────────────┘                    └──────────────────┘
                       │                                       │
                       └──────────────────────────────────────┘
                                          │
                                          ▼
                              ┌──────────────────┐
                              │ Media Payload JSON │
                              │ {type:"media",     │
                              │  url, key, iv}     │
                              └──────────────────┘
                                          │
                                          ▼
                              ┌──────────────────┐
                              │ Signal Encrypt     │
                              │ (Double Ratchet or │
                              │  Sender Keys)      │
                              └──────────────────┘
                                          │
                                          ▼
                              ┌──────────────────┐
                              │ Send as normal msg │
                              │ (WS or REST)       │
                              └──────────────────┘
```

Key insight: The media payload (URL + decryption key + IV) is itself encrypted by the Signal Protocol. Even if someone intercepts the R2 URL, the blob is AES-256-GCM encrypted. And even if they get the blob, they need the key which was Signal-encrypted.

6.2 Media Decryption (Receiver)

1. Signal-decrypt the message → get JSON payload `{type:"media", url, key, iv}`
2. Fetch encrypted blob from Cloudflare R2 (public URL, no auth)
3. Import AES-256-GCM key from base64
4. `crypto.subtle.decrypt({name:'AES-GCM', iv}, key, cipherBuf)`
5. Convert decrypted bytes to `...` URI
6. Render in `<Image>` component

7 Real-Time Messaging (WebSocket)

7.1 Phoenix Channel Architecture

```
CLIENT                                        SERVER (Phoenix)
──────                                        ────────────────

WebSocket Connect ──────────────────────────► Accept (JWT auth)
wss://nano-synapsys-server.fly.dev
     /socket/websocket?token={JWT}

phx_join("room:123") ───────────────────────► Join room:123 channel
◄──────────────────────────── phx_reply(ok)

phx_join("group:456") ──────────────────────► Join group:456 channel
◄──────────────────────────── phx_reply(ok)

new_message({content}) ─────────────────────► Broadcast to room:123
◄──── new_message({id, from_user, content})   Echo back to all

group_message({content}) ───────────────────► Broadcast to group:456
◄─ group.message({id, from_user, content})    Echo to all members

heartbeat (every 30s) ──────────────────────► Keep alive
◄──────────────────────────── phx_reply(ok)

RECONNECTION:
on close → exponential backoff (1s → 2s → 4s → ... → 30s max)
on reconnect → delta-sync all conversations (syncOnConnect)
```

7.2 Message Delivery Strategy

Scenario	Transport	Fallback
Text DM (WS open)	Phoenix WebSocket	—
Text DM (WS closed)	REST POST /api/messages	—
Image DM (any)	Always REST (too large for WS)	—
Group text (WS open)	Phoenix WebSocket	REST POST
Reconnect	Delta-sync all convos	Direct API fetch

8 Local Database & Sync

8.1 SQLCipher Schema

```
┌──────────────────────────────────────────────────────────────────┐
│  SQLCipher Database: synaptyc.db                                 │
│  Encryption: 256-bit key stored in SecureStore                   │
│  Key: synaptyc_db_key_v1 (hex-encoded random bytes)              │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │  TABLE: messages                                         │    │
│  ├──────────────┼──────────┼──────────────────────────────┤    │
│  │ id           │ INTEGER  │ PRIMARY KEY (server message ID)  │    │
│  │ convo_key    │ TEXT     │ 'dm_123' or 'group_456'         │    │
│  │ from_id      │ TEXT     │ sender user ID                   │    │
│  │ content      │ TEXT     │ decrypted plaintext              │    │
│  │ is_media     │ INTEGER  │ 1 if media payload               │    │
│  │ media_url    │ TEXT     │ R2 URL (for indexing)            │    │
│  │ created_at   │ TEXT     │ ISO 8601 timestamp               │    │
│  └──────────────┴──────────┴──────────────────────────────┘    │
│  INDEX: idx_msg_convo (convo_key, created_at)                    │
│  INDEX: idx_msg_content (content) — full-text search             │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │  TABLE: sync_cursors                                     │    │
│  ├──────────────┼──────────┼──────────────────────────────┤    │
│  │ convo_key    │ TEXT     │ PRIMARY KEY                      │    │
│  │ last_msg_id│ INTEGER  │ High-water mark (newest msg ID)  │    │
│  └──────────────┴──────────┴──────────────────────────────┘    │
└──────────────────────────────────────────────────────────────────┘
```

8.2 Two-Phase Load Strategy

1. **Phase 1 (instant):** Load cached messages from local SQLCipher DB → display immediately
2. **Phase 2 (background):** Delta-sync from server (`GET /api/messages/{id}?after_id={cursor}`) → decrypt → upsert to DB → refresh UI

This ensures the user sees messages instantly even on slow networks.

8.3 Delta Sync

Each conversation tracks a `last_msg_id` cursor. On sync, only messages with ID > cursor are fetched. After decrypt and persist, the cursor advances to the new max ID.

9 Banner AI Assistant

9.1 Architecture

Banner is an AI-powered assistant with **permission-gated** access to device capabilities. All inference happens server-side via `POST /api/bot/chat` .

9.2 Permission Model

Permission	Key	Grants Access To
Read Messages	`banner_perm_msgs`	Message history from conversations
Send Messages	`banner_perm_send`	Send messages to contacts on user's behalf
Calendar	`banner_perm_cal`	Read events (7-day window), create events
Contacts	`banner_perm_con`	List phone contacts (up to 300)

9.3 Action System

Banner can embed structured actions in its response using `<<<ACTION:{...}>>>` blocks. Each action requires explicit user confirmation before execution.

Action Type	Payload	Effect
`send_message`	`{to_username, content}`	Send encrypted DM to contact
`create_event`	`{title, start, end, notes}`	Create calendar event
`send_sms`	`{to_phone, to_name, content}`	Open SMS composer

10 Backend API Reference

Base URL: `https://nano-synapsys-server.fly.dev`

10.1 Authentication

Method	Endpoint	Body	Response
POST	/auth/register	{username, device_id, pin}	{token, user, refresh}
POST	/auth/pin-login	{device_id, pin}	{token, user, refresh}
POST	/auth/pin-reset	{device_id, username, new_pin}	{token, user, refresh}
POST	/auth/bio-refresh	{refresh}	{token, refresh, user}
GET	/auth/me	—	{user}

10.2 Messaging

Method	Endpoint	Body / Query	Description
GET	/api/users	—	List all users
POST	/api/messages	{to, content}	Send DM
GET	/api/messages/{peerId}	?after_id=N	Fetch DM history (delta)
DELETE	/api/messages/delete/{userId}	—	Delete conversation
GET	/api/groups	—	List groups
POST	/api/groups	{name, description}	Create group
GET	/api/groups/{id}/messages	?after_id=N	Fetch group messages
POST	/api/groups/{id}/messages	{content}	Send group message
DELETE	/api/groups/{id}	—	Delete group (admin)
POST	/api/groups/{id}/leave	—	Leave group

10.3 Signal Protocol / Keys

Method	Endpoint	Body	Description
POST	/api/signal/prekeys	{identity_key, signed_prekey_id, signed_prekey, signed_prekey_signature}	Upload pre-key bundle
GET	/api/signal/prekeys/{userId}	—	Fetch peer's pre-key bundle
GET	/api/signal/prekeys/count	—	Check remaining one-time keys
POST	/api/signal/prekeys/one-time	{one_time_prekeys: [{id, key}]}	Upload one-time pre-keys
POST	/api/signal/sender-keys	{group_id, keys: [{recipient_id, encrypted_sk}]}	Distribute group sender keys
GET	/api/signal/sender-keys/{groupId}	—	Fetch group sender keys

10.4 Media & Other

Method	Endpoint	Body	Description
POST	/api/media/presign	{mime}	Get presigned R2 upload URL
POST	/api/push-token	{token}	Register Expo push token
PATCH	/api/profile	{display_name, avatar_base64, ...}	Update profile
POST	/api/bot/chat	{message, image_base64, permissions, ...}	Banner AI inference
GET	/api/contacts	—	List accepted contacts
GET	/api/invites/validate/{token}	—	Validate invite token

11 Security Model

11.1 Threat Model

Threat	Mitigation
Server compromise	E2EE — server never sees plaintext. Keys stored only on device.
Network interception	TLS (HTTPS/WSS) + Signal Protocol encryption layer.
Device theft (locked)	Face ID + PIN required. SQLCipher DB encrypted at rest.
Device theft (unlocked)	24h skip-auth limits exposure window. Keys in SecureStore (Keychain).
Media interception	AES-256-GCM encryption before upload. Key distributed via Signal.
Key compromise	Forward secrecy — Double Ratchet ensures past messages unrecoverable.
Replay attack	Unique nonces per message. Chain key advances prevent reuse.
Out-of-order delivery	Skipped message key cache (up to 1000) handles reordering.

11.2 What the Server Knows

- Who is messaging whom (metadata: sender ID, recipient ID, timestamps)
- Message sizes (encrypted blob length)
- Group membership lists
- Public keys (identity, signed pre-keys, one-time pre-keys)

11.3 What the Server Does NOT Know

- Message content (all messages are E2EE)
- Media content (AES-256-GCM encrypted, key distributed via Signal)
- User PINs (hashed server-side, never stored in plaintext)
- Private keys (generated and stored only on device)

11.4 Disappearing Messages

Messages with a `disappear_after` value are deleted client-side after the timer expires. The server may also enforce server-side deletion. Timer options: 1 min, 5 min, 10 min, 5 days, 10 days, 30 days.

12 Third-Party Integrations

Service	Purpose	Data Shared
Cloudflare R2	Encrypted media storage	AES-256-GCM ciphertext only (opaque blobs)
Expo Push	Push notification routing	Expo push token, notification payload
Fly.dev	Backend hosting	API requests (encrypted payloads)
Apple (Face ID)	Biometric authentication	None (on-device only)
Apple (Keychain)	SecureStore backend	Encrypted key-value pairs

13 Data Flow Diagrams

13.1 DM Message Send → Receive (Complete Flow)

```
ALICE                          SERVER                          BOB
─────                          ──────                          ───
1. Type message
   │
2. encryptDM(plaintext)
   ├─ Get/build Signal session
   ├─ Double Ratchet encrypt
   └─ Wrap in envelope:
      {sig:true, v:1,
       hdr:{spk, n, pn},
       ct:base64(nonce+ct)}
   │
3. WS: phx_send("room:bob",  ──────────►  4. Receive encrypted
       "new_message",                        envelope (opaque)
       {content: envelope})                  │
                                          5. Broadcast to Bob
                                   ◄──       via room:bob channel
                                             │
                                             ▼
                                          6. Bob receives via WS
                                             │
                                          7. decryptDM(envelope)
                                             ├─ Parse Signal envelope
                                             ├─ Get/build session
                                             │  (X3DH if first msg)
                                             ├─ Double Ratchet decrypt
                                             └─ Return plaintext
                                             │
                                          8. Display in UI
                                          9. Persist to SQLCipher DB
                                         10. Update sync cursor
```

13.2 App Initialization Flow

```
App Launch
   │
   ▼
Register Push Notifications
   │
   ▼
Check SecureStore for JWT ─── NO ──► Show Auth Screen (PIN)
   │                                          │
   YES                                ┌────────┼────────┐
   │                                  │        │        │
   ▼                              Login   Register   Reset
Check 24h Skip Window                │        │        │
   │                                 └────────┼────────┘
┌──┴──┐                                       │
│YES  │NO                                     ▼
│     │                                POST /auth/{endpoint}
│     ▼                                       │
│  Bio enabled? ── NO ──► Validate ──────────►│
│     │                    stored             │
│    YES                   token              │
│     │                    GET /auth/me       │
│     ▼                        │              │
│  Face ID                     ▼              │
│  Prompt ◄──────────── {token, user, refresh}  │
│     │                        │              │
│     ▼                        │              │
│  bio-refresh                 │              │
│  (silent) ◄──────────────────┘              │
│     │                                       │
└──┬──┘                                       │
   │                                          │
   ▼                                          ▼
HomeScreen ◄──────────────────────────── Save to SecureStore
   │
   ├─ initE2EE(token)         ← Generate/upload Signal pre-keys
   ├─ registerPushToken()     ← Upload Expo push token
   ├─ connectWS()             ← Open Phoenix WebSocket
   └─ Render tabs
```